

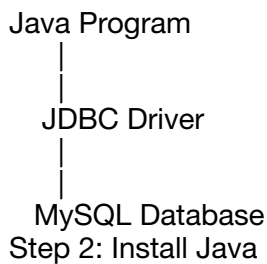
Step 1: What is JDBC?

JDBC (Java Database Connectivity) is an API that allows Java programs to communicate with databases such as MySQL, Oracle, PostgreSQL, etc.

Using JDBC, we can

- Create a database
- Create tables
- Insert records
- Update records
- Delete records
- Display records

Flow:



Step 2: Install Java

Check whether Java is installed.

```
java -version
```

and

```
javac -version
```

If not installed,

Ubuntu

```
sudo apt update
sudo apt install default-jdk
```

Verify

```
java -version
```

Step 3: Install MySQL

Ubuntu

```
sudo apt install mysql-server
```

Start MySQL

```
sudo systemctl start mysql
```

Login

```
sudo mysql
```

or

```
mysql -u root -p
```

Step 4: Create Database

Inside MySQL

```
CREATE DATABASE studentdb;
```

Check

```
SHOW DATABASES;
```

Use it

```
USE studentdb;
```

Step 5: Download JDBC Driver

Download MySQL Connector/J.

Example

mysql-connector-j-9.x.x.jar

Suppose it is saved in

~/jdbc/lib/

Project structure

JDBCProject

```
|
|--src
|   |
|   |--StudentCRUD.java
|
|--lib
|   |
|   |--mysql-connector-j-9.x.x.jar
```

Step 6: Create Table

Login to MySQL

```
USE studentdb;
```

Create table

```
CREATE TABLE student(
id INT PRIMARY KEY,
name VARCHAR(50),
branch VARCHAR(20),
marks INT
);
```

Check

```
DESC student;
```

Step 7: Write Java Program

```
import java.sql.*;
```

```
public class StudentCRUD {

    static final String URL =
```

```

        "jdbc:mysql://localhost:3306/studentdb";

static final String USER = "root";

static final String PASSWORD = "root";

public static void main(String args[]) {

    try {

        Class.forName("com.mysql.cj.jdbc.Driver");

        Connection con =
            DriverManager.getConnection(URL, USER, PASSWORD);

        Statement st = con.createStatement();

        // INSERT

        st.executeUpdate(
            "INSERT INTO student VALUES(101,'Rahul','CSE',90)");

        st.executeUpdate(
            "INSERT INTO student VALUES(102,'Amit','IT',88)");

        System.out.println("Records Inserted");

        // UPDATE

        st.executeUpdate(
            "UPDATE student SET marks=95 WHERE id=101");

        System.out.println("Record Updated");

        // DELETE

        st.executeUpdate(
            "DELETE FROM student WHERE id=102");

        System.out.println("Record Deleted");

        // DISPLAY

        ResultSet rs =
            st.executeQuery("SELECT * FROM student");

        while(rs.next()){

            System.out.println(
                rs.getInt("id")+" "
                + rs.getString("name")+" "
                + rs.getString("branch")+" "
                + rs.getInt("marks"));

        }

        con.close();

    }

    catch(Exception e){

```

```

        System.out.println(e);
    }
}
}

```

Step 8: Compile

Go inside project

```
cd JDBCProject
```

Compile

```
javac -cp "lib/mysql-connector-j-9.x.x.jar" src/StudentCRUD.java
```

Step 9: Run

```
java -cp "src:lib/mysql-connector-j-9.x.x.jar" StudentCRUD
```

On Windows, replace : with ; in the classpath:

```
java -cp "src;lib/mysql-connector-j-9.x.x.jar" StudentCRUD
```

Expected Output

Records Inserted

Record Updated

Record Deleted

101 Rahul CSE 95

Verify in MySQL

```
SELECT * FROM student;
```

Output

```

+-----+-----+-----+-----+
| id | name | branch | marks |
+-----+-----+-----+-----+
|101 |Rahul |CSE    |95    |
+-----+-----+-----+-----+

```

JDBC Classes Used

Class	Purpose
DriverManager	Creates database connection
Connection	Represents connection to database
Statement	Executes SQL queries
ResultSet	Stores SELECT query results
SQLException	Handles database errors

JDBC Execution Flow

Start

```

|
| Load JDBC Driver
|

```

```

|
| Create Connection
|

```

```

|
| Create Statement
|

```

```

|
| Execute SQL
|

```

```

|
| Process ResultSet
|

```

```

|
| Close Connection
|

```

End
SQL Operations Used
Create Database
CREATE DATABASE studentdb;
Create Table
CREATE TABLE student(
id INT PRIMARY KEY,
name VARCHAR(50),
branch VARCHAR(20),
marks INT
);
Insert
INSERT INTO student VALUES(101,'Rahul','CSE',90);
Update
UPDATE student
SET marks=95
WHERE id=101;
Delete
DELETE FROM student
WHERE id=102;
Display
SELECT * FROM student;
Recommended Learning Order
Install Java (JDK)
Install MySQL Server
Learn basic SQL (CREATE, INSERT, UPDATE, DELETE, SELECT)
Download the MySQL JDBC driver (mysql-connector-j)
Learn JDBC concepts:
Loading the driver
Creating a Connection
Using Statement and PreparedStatement
Reading data with ResultSet
Build a CRUD application like the one above.
Move on to Servlets, JSP, and then frameworks such as Spring Boot.

Since I remember you've already set up a basic JDBC project on Windows and successfully compiled your first Java program, the next good exercise is to build a menu-driven JDBC CRUD application (using Scanner and PreparedStatement) where users can choose to Insert, Update, Delete, Search, or Display records interactively. This is a common university lab assignment and a solid stepping stone toward Java web development.